Atividade de avaliação individual 2

Professor Eng. Dr. Gerson Penha

Introdução

Prezado empreendedor, permita-me agora conduzi-lo por uma reflexão mais ampla sobre o ambiente tecnológico no qual seu sistema de logística reversa será construído. Esta introdução tem o propósito de situar sua decisão de projeto em um contexto histórico e conceitual, para que você compreenda não apenas o como, mas principalmente o porquê de cada escolha tecnológica que faremos.

A trajetória da internet, desde seus primórdios como uma rede de troca de mensagens entre instituições de pesquisa até a imensa teia global que conecta bilhões de dispositivos, é marcada por saltos qualitativos na maneira como construímos e entregamos software. No início da década de 1990, o surgimento do navegador gráfico, como o Mosaic e posteriormente o Netscape Navigator, transformou a internet de um domínio restrito a especialistas em uma plataforma acessível ao público em geral. Nesse período, as páginas web eram essencialmente documentos hipertexto estáticos, escritos em HTML, cuja função primordial era estruturar textos e imagens com links entre si. O HTML, ou HyperText Markup Language, fornecia os elementos de cabeçalho, parágrafo, lista e âncora, mas carecia completamente de qualquer capacidade de estilização ou interatividade. Foi então que o CSS, ou Cascading Style Sheets, emergiu como uma camada separada, permitindo que a apresentação visual fosse definida de modo consistente e separada do conteúdo estrutural. Essa separação de preocupações, ainda que rudimentar, já anunciava um princípio arquitetural que se tornaria recorrente: o de dividir responsabilidades para facilitar a manutenção e a evolução.

Contudo, a verdadeira revolução ocorreu com a padronização do JavaScript como linguagem de script no lado do cliente. Por volta do final dos anos 1990, os navegadores passaram a executar código JavaScript capaz de manipular o Document Object Model, ou DOM, que é a representação em memória da página HTML. Isso permitiu que os desenvolvedores criassem efeitos visuais, validações de formulário e atualizações parciais da página sem necessidade de recarregar todo o documento vindo do servidor. O termo Web 2.0, cunhado no início dos anos 2000, capturou esse espírito de aplicações mais interativas e centradas no usuário, com exemplos notáveis como o Gmail e o Google Maps, que surpreenderam o mundo ao oferecer uma experiência de uso contínua e responsiva, desafiando a noção de que a web seria apenas um repositório de páginas estáticas.

Esse movimento, no entanto, ainda estava limitado pelo modelo de requisição-resposta tradicional. Cada ação do usuário exigia uma nova requisição ao servidor, que devolvia uma página HTML inteiramente renderizada, com todo o markup, CSS e JavaScript embutidos. O servidor era responsável por gerar essa página, o que significava que a lógica de apresentação e a lógica de negócio residiam no mesmo ambiente, frequentemente em linguagens como PHP, JSP ou ASP.NET. Esse modelo, conhecido como Server-Side Rendering, funcionou bem por muitos anos, mas apresentava ineficiências evidentes: o tráfego de rede era elevado, a latência era perceptível e a experiência do usuário sofria com a recarga total da tela a cada interação. Foi então que arquitetos de software começaram a imaginar uma abordagem radicalmente diferente,

na qual o servidor não mais entregaria páginas prontas, mas sim dados brutos, geralmente em formato JSON, e o cliente seria responsável por construir a interface dinamicamente.

Dessa mentalidade nasceram as Single Page Applications, ou SPA, cujo conceito central é que toda a aplicação é carregada em uma única página HTML, e as navegações entre diferentes estados e funcionalidades ocorrem por meio de substituições de partes do DOM, sem recarregar a página inteira. Frameworks como Angular, React e Vue.js popularizaram essa abordagem, fornecendo mecanismos eficientes para gerenciar o estado da aplicação, renderizar componentes reutilizáveis e comunicar-se com APIs no servidor por meio de chamadas assíncronas, tipicamente utilizando XMLHttpRequest ou a mais moderna API Fetch. As SPA trouxeram benefícios notáveis em termos de responsividade e fluidez, pois apenas os dados necessários para a operação corrente são trafegados, e o cliente mantém um estado rico que pode ser atualizado instantaneamente. Isso, contudo, não veio sem custos: a complexidade do front-end aumentou exponencialmente, exigindo ferramentas de empacotamento como Webpack ou Vite, e a primeira carga da aplicação tornou-se mais pesada, já que todo o código JavaScript necessário para todas as funcionalidades precisava ser baixado antes de a aplicação se tornar interativa. Além disso, o gerenciamento de estado em aplicações de grande porte tornou-se um desafio por si só, com bibliotecas como Redux ou MobX sendo empregadas para manter a previsibilidade das mutações.

Paralelamente a essa evolução no lado do cliente, o lado do servidor também passou por transformações profundas. As linguagens dinâmicas, como Ruby e Python, ganharam popularidade por sua agilidade no desenvolvimento, mas a necessidade de desempenho e escalabilidade trouxe o Node.js para o centro do debate. O Node.js, ao utilizar o motor V8 do Chrome e um modelo de I/O não bloqueante orientado a eventos, permitiu que o JavaScript, antes restrito ao navegador, se tornasse uma linguagem de servidor viável, especialmente para aplicações com alta intensidade de operações de entrada e saída. O TypeScript, por sua vez, adicionou uma camada de tipagem estática sobre o JavaScript, oferecendo segurança de tipos, melhor autocompletar em editores e detecção precoce de erros, o que se mostrou inestimável em projetos de média e grande escala. Essa combinação tornou-se a pilha preferida de inúmeras startups e empresas consolidadas, pois unifica a linguagem no cliente e no servidor, reduzindo a curva de aprendizado e facilitando a comunicação entre equipes.

No entanto, uma aplicação moderna não vive apenas de código de negócio; ela precisa interagir com dados persistentes de maneira eficiente e segura. É aqui que entra o conceito de ORM, ou Object-Relational Mapping. Historicamente, os bancos de dados relacionais dominam o armazenamento corporativo, com suas tabelas, colunas, chaves estrangeiras e consultas em SQL. Porém, a programação orientada a objetos lida com grafos de objetos, herança e polimorfismo, criando uma desconexão conceitual conhecida como impedance mismatch. Para resolver essa discrepância, os ORMs surgem como camadas de abstração que mapeiam tabelas para classes, linhas para objetos e relacionamentos para referências entre objetos. Dessa forma, o desenvolvedor pode escrever consultas em uma sintaxe orientada a objetos, como o Prisma Client, e o ORM se encarrega de traduzi-las para SQL otimizado, executar as operações e transformar os resultados novamente em objetos. O Prisma ORM, em particular, destaca-se por sua abordagem baseada em um arquivo de schema

declarativo, que define os modelos e seus relacionamentos, e a partir do qual o Prisma gera automaticamente um cliente de banco de dados com tipos TypeScript rigorosos. Isso elimina a necessidade de escrever consultas SQL manualmente, reduzindo erros de sintaxe e tornando o código mais legível e manutenível, além de fornecer migrações controladas e suporte a transações.

Ao combinar esses elementos, chegamos à arquitetura monolítica que adotaremos para a próxima fase do desenvolvimento do greencode, agora não mais como uma aplicação de linha de comando, mas como um servidor HTTP completo. Um monólito, neste contexto, não carrega a conotação pejorativa que por vezes se atribui a sistemas legados e rígidos; trata-se, antes, de uma escolha consciente para um estágio inicial do empreendimento, onde a simplicidade de implantação, a facilidade de depuração e a coesão do código são mais valiosas do que a distribuição de microsserviços. Com o Node.js, teremos um runtime leve e rápido; com o TypeScript, garantiremos a segurança dos contratos entre as camadas; e com o Prisma ORM, extrairemos o máximo de produtividade na camada de persistência, sem abrir mão da performance e da integridade referencial. O que antes era um sistema CLI que operava em arquivos de texto criptografados, agora se expande para uma aplicação que expõe uma API RESTful, que será consumida por um front-end SPA, e que armazena seus dados em um banco de dados relacional como PostgreSQL ou MySQL. Essa evolução não é apenas técnica, mas também estratégica, pois permite que o sistema atenda a múltiplos usuários simultaneamente, que ofereça interfaces mais ricas e que se integre a outros sistemas por meio de APIs padronizadas, preparando o terreno para a escalabilidade e a adoção por um número crescente de organizações clientes.

Atividade

Prezado empreendedor, chegamos ao momento de transição da arquitetura do sistema greencode, migrando de uma aplicação de linha de comando com persistência em arquivos criptografados para um servidor monolítico robusto, que expõe uma API e gerencia seus dados em um banco de dados relacional. Esta segunda atividade é o coração da evolução técnica do seu empreendimento, pois é nela que você consolidará as abstrações orientadas a objetos que projetou anteriormente, agora com a infraestrutura adequada para atender a múltiplos usuários simultâneos e a um volume crescente de transações. O direcionamento que ofereço a seguir não é um roteiro rígido, mas um conjunto de princípios e sugestões que visam estimular sua criatividade arquitetural, deixando a você a liberdade de explorar soluções próprias, desde que alinhadas aos fundamentos da engenharia de software.

O primeiro aspecto a considerar é a estruturação do projeto em camadas. Embora o sistema CLI já possuísse uma separação entre domínio, serviços e infraestrutura, a migração para o ambiente servidor exige uma reorganização mais refinada. Sugiro que você adote uma arquitetura em camadas inspirada no padrão Controller-Service-Repository, que é amplamente difundido em aplicações monolíticas. A camada de controllers será responsável por receber as requisições HTTP, interpretar os parâmetros e delegar a execução aos serviços apropriados. A camada de serviços conterá toda a lógica de negócio, incluindo as validações, as regras de transição de status dos equipamentos e os cálculos financeiros. A camada de repositórios, por sua vez, será implementada utilizando o Prisma Client, que atuará como a ponte entre o seu modelo

de domínio e as tabelas do banco de dados. Essa separação não é meramente estética; ela garante que sua lógica de negócio permaneça imune a mudanças na interface de comunicação, seja ela HTTP, CLI ou futuramente mensageria.

Para a implementação do servidor HTTP, você terá à sua disposição o framework Fastify, que recomendo em detrimento do Express por sua performance superior, suporte nativo a esquemas de validação com JSON Schema e tratamento de erros mais estruturado. O Fastify permite que você defina rotas de forma declarativa, com hooks para autenticação e logging, e sua arquitetura baseada em plugins facilita a organização modular do código. Alternativamente, se preferir uma abordagem mais minimalista, o próprio Express continua sendo uma escolha válida, mas sugiro que você avalie cuidadosamente as necessidades de escalabilidade e a complexidade das validações antes de decidir. Independentemente do framework escolhido, você deverá estruturar seu servidor para iniciar em uma porta configurável por variável de ambiente, com suporte a graceful shutdown para garantir que as conexões ativas sejam finalizadas adequadamente durante reinicializações.

A camada de persistência, agora baseada em Prisma ORM, exigirá que você replique o modelo de domínio em um arquivo de schema.prisma, definindo os modelos de Organizacao, Contrato, Lote, Equipamento e Movimentacao com seus respectivos campos, tipos de dados e relacionamentos. Atenção especial deve ser dada aos relacionamentos: um Organizacao possui um Contrato opcional e muitos Lotes; um Lote possui muitos Equipamentos; um Equipamento possui muitas Movimentacoes. O Prisma gerencia essas associações por meio de chaves estrangeiras implícitas, e você pode aproveitar os recursos de eager loading e seleção de campos para otimizar as consultas. Além disso, o Prisma oferece migrações versionadas, que permitem que você evolua seu esquema de banco de dados de forma controlada, criando arquivos de migração que descrevem as alterações em SQL. Recomendo que você utilize o PostgreSQL como banco de dados alvo, devido à sua maturidade, suporte a transações ACID e tipos de dados avançados como JSON e UUID, que podem ser úteis para armazenar dados semiestruturados como as cláusulas dos contratos.

Um dos desafios centrais desta atividade será a adaptação do sistema de autenticação. No CLI, você implementou um mecanismo baseado em sessões com expiração e hash de senhas. Agora, em um ambiente de API RESTful stateless, é mais apropriado adotar um esquema de autenticação baseado em tokens JWT, ou JSON Web Tokens. Você pode integrar o Fastify com o plugin @fastify/jwt, que fornece métodos para assinar e verificar tokens de forma simples e segura. O fluxo de login deve receber as credenciais, verificar o hash da senha armazenado no banco de dados (utilizando a mesma biblioteca de criptografia que você empregou no CLI, como bcrypt) e, em caso de sucesso, gerar um token JWT contendo o identificador do usuário e seu papel. Esse token deve ser enviado ao cliente e incluído no cabeçalho Authorization de todas as requisições subsequentes. Um middleware global ou um hook do Fastify pode ser responsável por validar o token em cada requisição e injetar as informações do usuário no objeto de requisição, permitindo que os serviços de negócio consultem o papel do usuário para aplicar as regras de autorização.

Outro ponto que merece reflexão é o tratamento de transações e concorrência. No sistema CLI, as operações eram atômicas em nível de arquivo, mas agora múltiplas

requisições podem acessar os mesmos registros simultaneamente. O Prisma oferece suporte a transações por meio do método $transaction, que permite agrupar múltiplas operações de leitura e escrita em uma única unidade atômica. Você deve identificar quais operações de negócio exigem transações, como a criação de um lote com a adição de vários equipamentos, ou a atualização do status de um equipamento com o registro de uma movimentação. Nessas situações, é fundamental que todas as alterações sejam confirmadas juntas ou nenhuma seja persistida, para manter a integridade referencial do sistema. Além disso, você deve considerar a implementação de um mecanismo de versionamento otimista para evitar atualizações perdidas, adicionando um campo versão em cada modelo e verificando-o antes de cada atualização.

A migração de dados do sistema CLI para o novo banco de dados também requer planejamento. Embora você possa reinserir manualmente os dados de teste, é mais interessante desenvolver um script de seed que popule o banco com uma massa de dados representativa, incluindo organizações, contratos, lotes e equipamentos com estados variados. Esse script pode ser executado durante a inicialização do ambiente de desenvolvimento ou por meio de um comando específico do Prisma. Os dados devem refletir os cenários de validação que você implementou no CLI, para que você possa verificar se as regras de negócio foram corretamente transportadas para a nova arquitetura. Aproveite essa oportunidade para revisar as validações e aprimorá-las, adicionando restrições de unicidade no banco de dados para campos como CNPJ e codigoBarrasInterno, e utilizando triggers ou constraints para garantir que estados inconsistentes não sejam persistidos.

Do ponto de vista da arquitetura de software, esta atividade é também uma oportunidade para você refletir sobre os limites da aplicação monolítica. Ao final desta atividade, você terá transformado o greencode em uma aplicação servidora plenamente funcional, com uma API documentada, um banco de dados normalizado e uma estrutura de código que reflete as melhores práticas de engenharia de software. Mais importante do que o código, contudo, é a capacidade de justificar as decisões arquiteturais tomadas, os trade-offs considerados e as alternativas rejeitadas, pois é essa capacidade analítica que distinguirá o seu empreendimento no mercado.

Por fim, para que o sistema possa ser comercializado com facilidade ele de ser compatível com as plataformas Windows 10 ou superior, Linux Ubuntu 24.04.03 ou superior e distribuições Linux derivadas do Ubuntu.